Министерство образования и науки Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования
САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ «ИТМО»
(ФГАОУ НИУ ИТМО)

ФПИиКТ

Вычислительная Математика

Лабораторная работа № 1

Решение системы линейных алгебраических уравнений СЛАУ

Вариант № 9

Выполнил:
Никитенко М. О.
студент группы P3214

Проверил:
Лузин Б. Е.

Санкт-Петербург, 2026

Оглавление

Задание **3**
Диаграмма классов ..... Ошибка! Закладка не определена.
Исходный код ..... Ошибка! Закладка не определена.
Результат работы программы ..... Ошибка! Закладка не определена.
Заключение **13**

Задание

3. В программе численный метод должен быть реализован в виде отдельной подпрограммы/метода/класса, в который исходные/выходные данные передаются в качестве параметров.
4. Размерность матрицы n<=20 (задается из файла или с клавиатуры - по выбору конечного пользователя).
5. Должна быть реализована возможность ввода коэффициентов матрицы, как с клавиатуры, так и из файла (по выбору конечного пользователя).

<u>Для итерационных методов должно быть реализовано:</u>

- Точность задается с клавиатуры/файла,
- Проверка диагонального преобладания (в случае, если диагональное преобладание в исходной матрице отсутствует, сделать перестановку строк/столбцов до тех пор, пока преобладание не будет достигнуто). В случае невозможности достижения диагонального преобладания - выводить соответствующее сообщение.
- Вывод нормы матрицы (любой, на Ваш выбор),
- Вывод вектора неизвестных: $x_1,\ x_2,\ \ldots, x_n$,
- Вывод количества итераций, за которое было найдено решение,
- Вывод вектора погрешностей: $|x_i^{(k)} - x_i^{(k-1)}|$.

- При некорректном пользовательском вводе (введена строка, не являющаяся именем константы в enum'е; введена строка вместо числа; введенное число не входит в указанные границы и т.п.) должно быть показано сообщение об ошибке и предложено повторить ввод поля.
- Для ввода значений null использовать пустую строку.
- Поля с комментарием "Значение этого поля должно генерироваться автоматически" не должны вводиться пользователем вручную при добавлении.

Описание хранимых в коллекции классов:

```
public class Product {
    private long id; //Значение поля должно быть больше 0, Значение этого поля должно быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDateTime creationDate; //Поле не может быть null, Значение этого поля должно генерироваться автоматически
    private int price; //Значение поля должно быть больше 0
    private UnitOfMeasure unitOfMeasure; //Поле может быть null
    private Person owner; //Поле не может быть null
}
public class Coordinates {
    private Long x; //Значение поля должно быть больше -852, Поле не может быть null
    private Integer y; //Поле не может быть null
}
public class Person {
    private String name; //Поле не может быть null, Строка не может быть пустой
    private float height; //Значение поля должно быть больше 0
    private Color eyeColor; //Поле не может быть null
    private Color hairColor; //Поле не может быть null
    private Country nationality; //Поле не может быть null
    private Location location; //Поле может быть null
}
public class Location {
    private long x;
    private Double y; //Поле не может быть null
    private double z;
}
public enum UnitOfMeasure {
    METERS,
    CENTIMETERS,
    SQUARE_METERS,
    LITERS,
    MILLILITERS;
}
public enum Color {
    BLACK,
    BLUE,
    ORANGE;
}
public enum Color {
    GREEN,
    RED,
    WHITE;
}
```

Описание метода

Сам метод простой итерации заключается в том, что мы постепенно итеративно строим приближения до достижения нужной точности (критерий нашей остановки алгоритма).

Сначала мы должны убедится что решение СЛАУ вообще существует, иначе дальнейшие итерации не имеют смысла. Для этого первым шагом проверяем $detA \neq 0$. После чего, проверяется условие сходимости итерационного процесса:

$$|a_{ij}| \geq \sum_{j \neq i} |a_{ij}|, \qquad i = 1,2, \dots, n$$

Самое главное, для одного из уравнений неравенство должно выполняться строго. После чего выражаем из каждого i-ого уравнения неизвестную x_i и строим вектор d – правых частей. Проверяем норму на условие сходимости:

$$||C_1|| = \max_{1 \leq i \leq n} \sum_{j=1}^{n} |c_{ij}| < 1$$

Если условие выполняется то задаем начальное приближение: $\mathrm{x}_0 = d$

После строим итеративно, следующие приближения: $x^{k+1} = Cx^k + d$

И проверяем на точность приближения: $\max_{1 \leq i \leq n} (|x_i^{k-1} - x_i^0|) < \epsilon$ – до этого момента и будут продолжаться итерации.

Листинг программы

```java
private boolean diagonalDominant() {  1 usage  Матвий
    for(int i = 0; i < this.matrixSize; i++) {
        int maxRow = i;
        double maxElement = Math.abs(this.matrix[i][i]);
        for(int j = i + 1; j < this.matrixSize; j++) {
            if(Math.abs(this.matrix[j][i]) > maxElement) {
                maxElement = Math.abs(this.matrix[j][i]);
                maxRow = j;
            }
        }
        if(maxRow != i) {
            Double[] tmpMatrix = matrix[i];
            matrix[i] = matrix[maxRow];
            matrix[maxRow] = tmpMatrix;

            double tmpD = dVector[i];
            dVector[i] = dVector[maxRow];
            dVector[maxRow] = tmpD;
        }
    }
    boolean isOneStrict = false;
    for(int i = 0; i < this.matrixSize; i++) {
        double summ = 0;
        for(int j = 0; j < this.matrixSize; j++) {
            if(i != j)  {
                summ += Math.abs(this.matrix[i][j]);
            }
        }
        if(Math.abs(this.matrix[i][i]) < summ) {
            return false;
        }
        if(Math.abs(this.matrix[i][i]) > summ) {
            isOneStrict = true;
        }
    }
    System.out.println("Матрица после перестановок: ");
    for(int i = 0; i < this.matrixSize; i++) {
        for(int j = 0; j < this.matrixSize; j++) {
            System.out.format("%10.3f", this.matrix[i][j]);
        }
        System.out.println();
```

```java
    System.out.println("Матрица после перестановок: ");
    for(int i = 0; i < this.matrixSize; i++) {
        for(int j = 0; j < this.matrixSize; j++) {
            System.out.format("%10.3f", this.matrix[i][j]);
        }
        System.out.println();
    }
    return isOneStrict;
}

private double getMaxRowNorm() {  2 usages  Матвий
    double normal = 0.0;
    for(int i = 0; i < this.matrixSize; i++) {
        double summ = 0;
        for(int j = 0; j < this.matrixSize; j++) {
            summ += Math.abs(this.matrix[i][j]);
        }
        normal = Math.max(normal, summ);
    }
    return normal;
}

public Double[] multiplyMatrix(Double[][] matrixA,  Double[] matrixB) {  1 usage  Матвий
    Double[] matrixC = new Double[matrixA.length];
    for(int i = 0; i < matrixA.length; i++) {
        matrixC[i] = 0.0;
        for(int j = 0; j < matrixA[i].length; j++) {
            matrixC[i] += matrixA[i][j] * matrixB[j];
        }
    }
    return matrixC;
}

public Double[] sumMatrix(Double[] matrixA,  Double[] matrixB) {  1 usage  Матвий
    Double[] matrixC = new Double[matrixA.length];
    for(int i = 0; i < matrixA.length; i++) {
        matrixC[i] = matrixA[i] + matrixB[i];
    }
    return matrixC;
}
```

```java
private void magicFunction() {  1 usage  Матвий
    double diag = 0;
    for(int i = 0; i < this.matrixSize; i++) {
        diag = matrix[i][i];
        for(int j = 0; j < this.matrixSize; j++) {
            if(i == j) {
                this.matrix[i][j] = 0.0;
            } else {
                this.matrix[i][j] = -(this.matrix[i][j] / diag);
            }
        }
    }
    for(int i = 0; i < this.matrixSize; i++) {
        this.dVector[i] /= diag;
    }
    System.out.println("Матрица после вывода коэффициентов");
    for(int i = 0; i < this.matrixSize; i++) {
        for(int j = 0; j < this.matrixSize; j++) {
            System.out.format("%10.3f", this.matrix[i][j]);
        }
        System.out.println();
    }
    System.out.println("Ваша вектор D: ");
    for (int i = 0; i < this.matrixSize; ++i) {
        System.out.format("%10.3f\n", this.dVector[i]);
    }
}
```

```java
public void startMethod() {  2 usages  Матвий
    if(determinant(matrix) < 1e-9) {
        System.out.println("Матрица вырождена, потому решений нет!");
        return;
    }
    if(!diagonalDominant()) {
        System.out.println("Диагонального преобладания в матрице нет :( плаки-плак");
        return;
    }
    this.magicFunction();
    if(getMaxRowNorm() >= 1.0) {
        System.out.println("Норма некорректная");
        return;
    }
    System.out.println("Норма матрицы: ||C_1|| = " + getMaxRowNorm());
    Double[] xStart = Arrays.copyOf(this.dVector, this.dVector.length);
    Double[] xLast;
    int k = 0;
    double maxDisp;
    do {
        k++;
        maxDisp= 0;
        Double[] xNew = sumMatrix(multiplyMatrix(this.matrix, xStart),  this.dVector);
        for(int i = 0; i < xStart.length; i++) {
            maxDisp = Math.max(maxDisp, Math.abs(xStart[i] - xNew[i]));
        }
        xLast = Arrays.copyOf(xStart, xStart.length);
        xStart = Arrays.copyOf(xNew, xNew.length);
    } while(maxDisp > this.eps);
    System.out.format("Ответ был найден спустя %d итераций\n", k);
    System.out.println("Вектор неизвестных: ");
    for(int i = 0; i < xLast.length; i++) {
        System.out.format("X_%d =  %10.6f\n", i+1, xStart[i]);
    }
    System.out.println("Вектор погрешностей");
    for(int i = 0; i < xLast.length; i++) {
        System.out.format("|X_%d^(%d) - X_%d^(%d)| =  %10.6f\n", i+1, k, i + 1, k-1, Math.abs(xLast[i] -  xStart[i]));
    }
}
```

Исходный код: https://github.com/YushisNoN/ComMathLab1

Пример работы программы

```
Выберите режим ввода:
[1]: Клавиатура
[2]: с Файла

-> 2
Система считывания файла активирована...
Некорректный формат ввода
Матрица после перестановок:
     3,000     2,000     1,000
     1,000     5,000     1,000
     2,000     1,000     3,000
Матрица после вывода коэффициентов
     0,000    -0,667    -0,333
    -0,200     0,000    -0,200
    -0,667    -0,333     0,000
Ваша вектор D:
     3,000
     2,333
     2,667
Норма некорректная
```

```
Выберите режим ввода:
[1]: Клавиатура
[2]: с Файла

-> 1
Введите размер матрицы n (0 < n < 21)

-> 3
Введите коэффициенты матрицы построчно через пробел
Введите коэффициенты 1 строки матрицы
-> 2 2 10
Введите коэффициенты 2 строки матрицы
-> 10 1 1
Введите коэффициенты 3 строки матрицы
-> 2 10 1
Ваша матрица:
     2,000     2,000    10,000
    10,000     1,000     1,000
     2,000    10,000     1,000
Хотите заново ввести матрицу?:
[1]: Ввести заново
[2]: Оставить текущую
-> 2
Введите элементы вектора неизвестных построчно
Введите коэффициенты 1 строки матрицы
-> 14
Введите коэффициенты 2 строки матрицы
-> 12
Введите коэффициенты 3 строки матрицы
-> 13
Ваша вектор D:
    14,000
    12,000
    13,000
```

```
Хотите заново ввести матрицу?:
[1]: Ввести заново
[2]: Оставить текущую
-> 2
Введите желаемую точность вычислений. (Например: 0.001):
-> 0.01
Матрица после перестановок:
    10,000     1,000     1,000
     2,000    10,000     1,000
     2,000     2,000    10,000
Матрица после вывода коэффициентов
     0,000    -0,100    -0,100
    -0,200     0,000    -0,100
    -0,200    -0,200     0,000
Ваша вектор D:
     1,200
     1,300
     1,400
Норма матрицы: ||C_1|| = 0.4
Ответ был найден спустя 5 итераций
Вектор неизвестных:
X_1 =    0,999568
X_2 =    0,999460
X_3 =    0,999316
Вектор погрешностей
|X_1^(5) - X_1^(4)| =    0,001932
|X_2^(5) - X_2^(4)| =    0,002460
|X_3^(5) - X_3^(4)| =    0,003084
```

Заключение

В процессе выполнения лабораторной работы, на практике был закреплен итеративный способ приближенных вычислений решений СЛАУ. Мною была разработанная и множественно раз исправлена программа для решения СЛАУ методом Простой итерации.